



HCMC University of Technology, VNU

Software Architecture



Chapter 2. SOLID Principles

Lecturer: Thai-Minh Truong, PhD.

Email: thaiminh@hcmut.edu.vn

CHAPTER 2. SOLID Principles



2.1. Single Responsibility Principle

2.2. Open-Closed Principle

2.3. Liskov Substitution Principle

2.4. Interface Segregation Principle

2.5. Dependency Inversion Principle

What is SOLID principles?



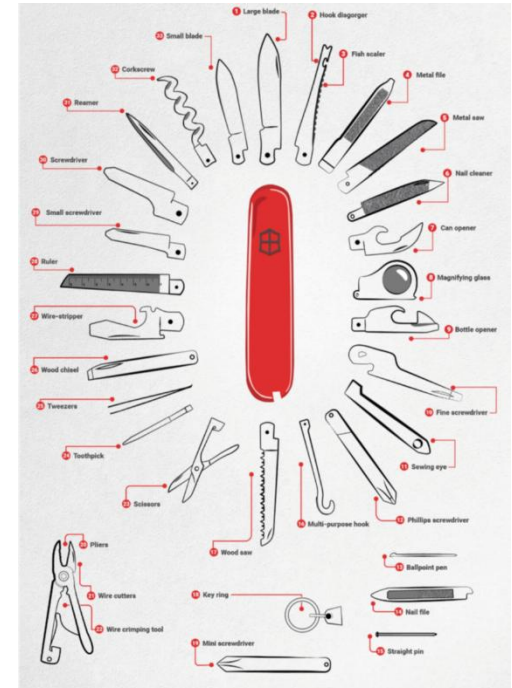
- Good software systems begin with **clean code**. On the one hand, if the bricks aren't well made, the architecture of the building doesn't matter much. On the other hand, you can make a substantial mess with well-made bricks. This is where the SOLID principles come in.
- **SOLID** principles tell us how to arrange our functions and data structures into classes, and how those classes should be interconnected.
- The goal of the principles is the creation of mid-level software structures that: *Tolerate change; Are easy to understand and Are the basis of components that can be used in many software systems.*

SRP: The Single Responsibility Principle

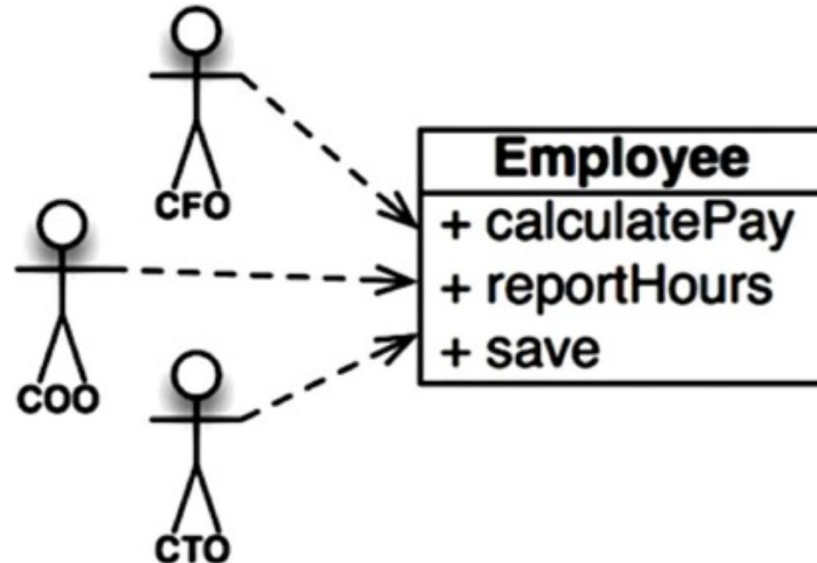


An active corollary to **Conway's law**: The best structure for a software system is heavily influenced by the social structure of the organization that uses it so that each software **module has one, and only one, reason to change.**

A module should be responsible to one, and only one, actor.



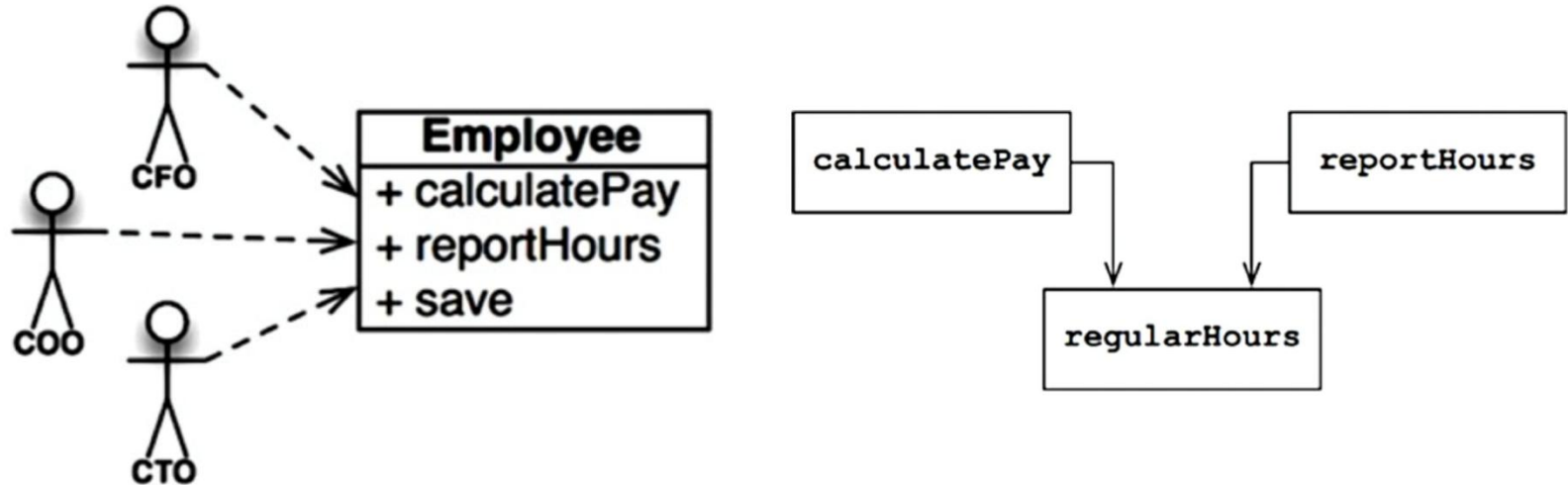
SRP: Case-study



The Employee class has 3 methods that reports to three different actors: CFO, COO & CTO. If one of these actors modifies their function, Employee is modified and the other actors are also affected.

=> Anti-pattern: *A module is responsible to more than actors.*

SRP: Case-study



Eg:

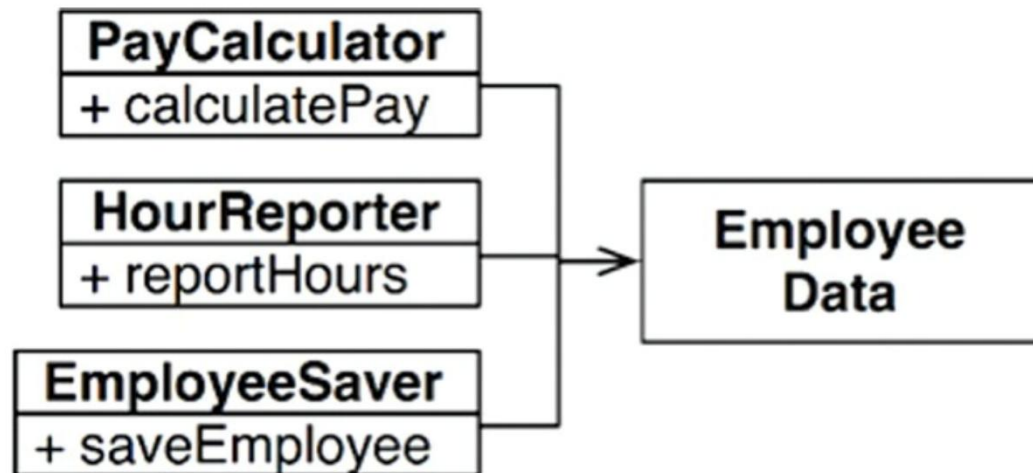
- Suppose that the *calculatePay()* function and the *reportHours()* function share a common algorithm for calculating non-overtime hours – *regularHours()*.
- Now, CFO's team decides that the way non-overtime hours are calculated needs to be tweaked (but COO's team don't want).
- Developers makes the required changes for CFO and they both don't know about COO also use this *reportHours()* for other purposes. Besides, COO's team even don't acknowledge their functions have been modified.

=> Ensure SRP: *A module should be responsible to one, and only one, actor.*

SRP: Solution



- We can refactor 3 methods into 3 separated classes that do not know about each other. These classes access to EmployeeData independently for retrieving employee data to perform their behaviors.



SRP: Solution



- The downside of this solution is that the developers now have three classes that they have to instantiate and track. A common solution to this dilemma is to use the **Facade pattern**

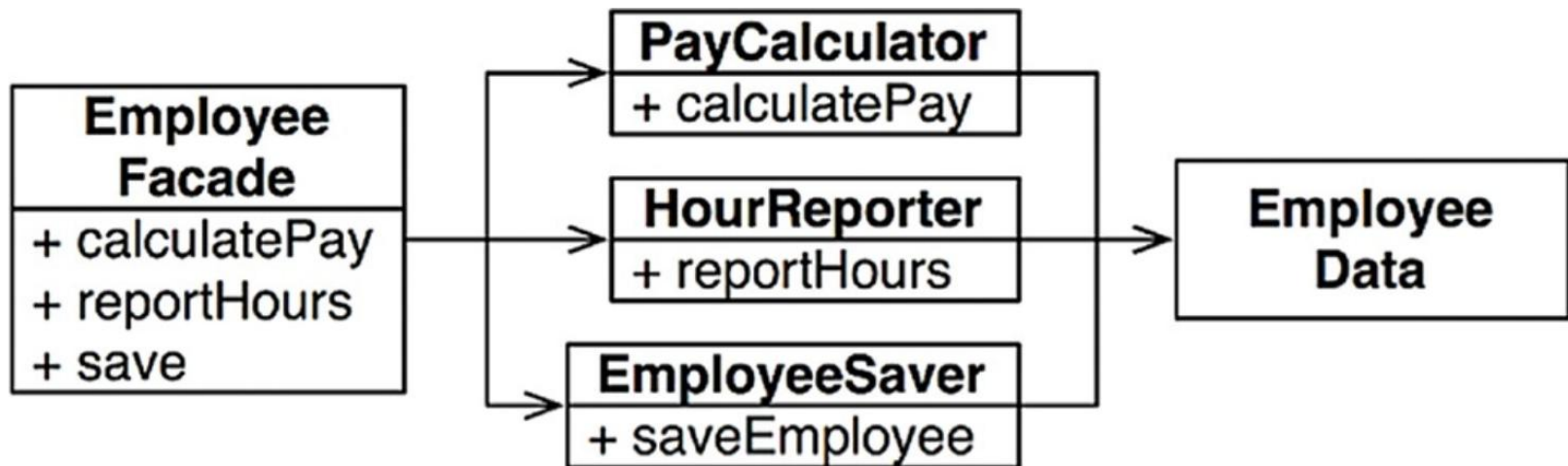
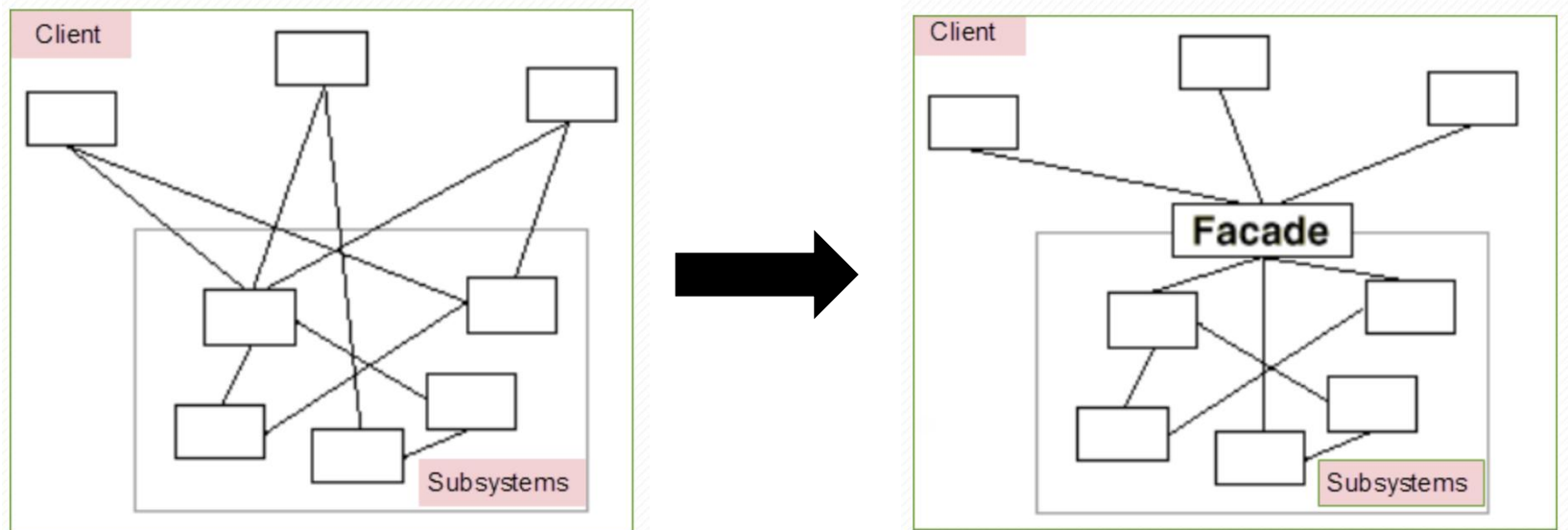


Figure 7.4 The Facade pattern

SRP: Solution – Facade pattern

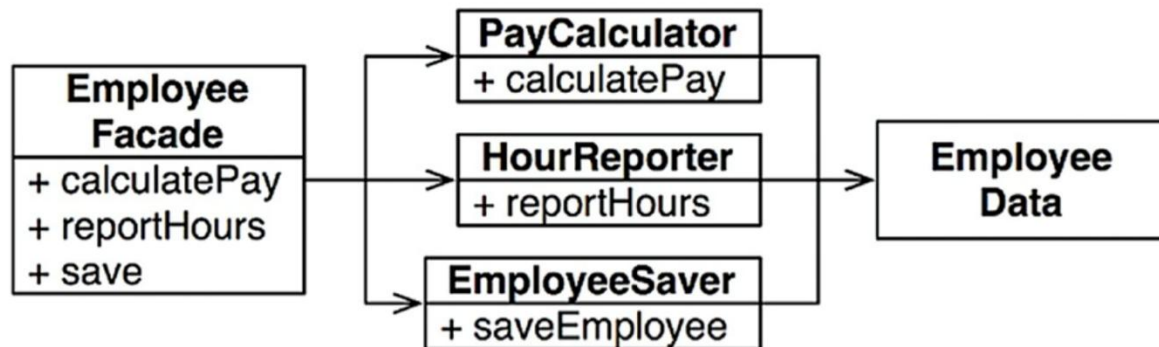


- **Facade** is a structural design pattern that provides a simplified (but limited) interface to a complex system of classes, library or framework.



- If the subsystems are complex, we can create many facades (but avoid too much and non-sense).

SRP: Example



```
public class EmployeeData {
    public Employee getEmployeeData();
}
```

```
public class PayCalculator {
    private EmployeeData empData;

    PayCalculator(EmployeeData empData){
        this.empData = empData;
    }

    public void calculatePay(){
        empData.getEmployeeData();
        //calculatePay();
    }
}
```

```
public class HourReporter {
    private EmployeeData empData;

    HourReporter(EmployeeData empData){
        this.empData = empData;
    }

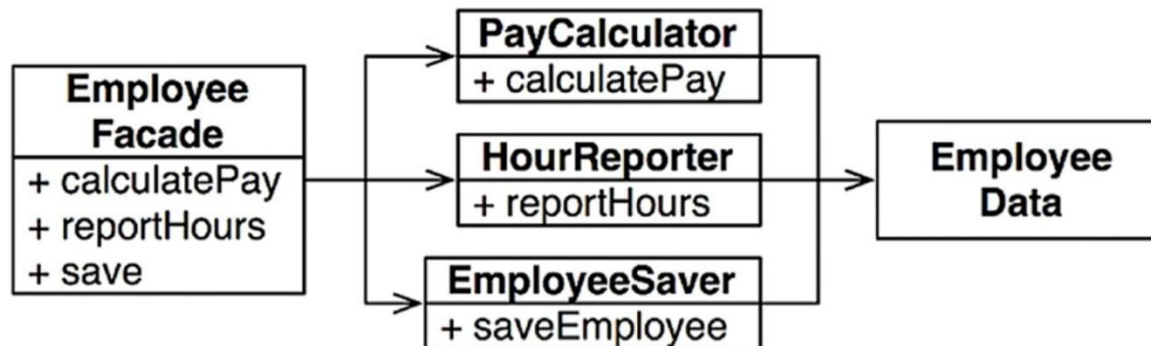
    public void reportHours(){
        empData.getEmployeeData();
        //reportHours();
    }
}
```

```
public class EmployeeSaver {
    private EmployeeData empData;

    EmployeeSaver(EmployeeData empData){
        this.empData = empData;
    }

    public void saveEmployee(){
        empData.getEmployeeData();
        //saveEmployee();
    }
}
```

SRP: Example



```
public class EmployeeFacade {
    private PayCalculator payCalculator;
    private HourReporter hourReporter;
    private EmployeeSaver employeeSaver;

    //Constructors
    EmployeeFacade(){
        payCalculator = new PayCalculator();
        hourReporter = new HourReporter();
        employeeSaver = new EmployeeSaver();
    }

    public void calculatePay(){
        payCalculator.calculatePay();
    }

    public void reportHours(){
        hourReporter.reportHours();
    }

    public void save(){
        employeeSaver.save();
    }
}
```

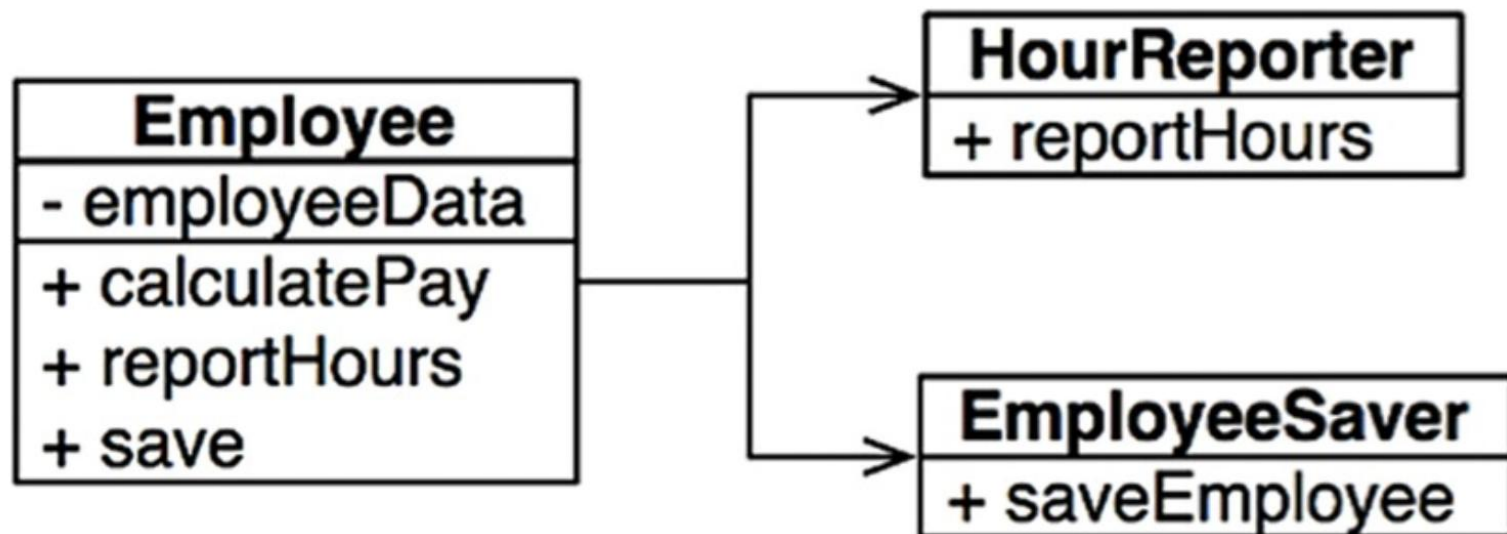
```
public class Main {
    public static void main(String[] args) {
        EmployeeFacade employeeFacade = new EmployeeFacade();

        employeeFacade.calculatePay();
        employeeFacade.reportHours();
        employeeFacade.save();
    }
}
```

SRP: Solution



- Some developers prefer to keep the most important business rules closer to the data. The most important method is kept in the original *Employee* class and used as *Facade* for the lesser functions.



SRP: Conclusion



- The Single Responsibility Principle is about functions and classes—but it reappears in a different form at two more levels. At the level of components, it becomes the **Common Closure Principle**. At the architectural level, it becomes the **Axis of Change** responsible for the creation of Architectural Boundaries.

SRP: Exercise 1



- Is there any problem with this design? If so, please refactor it.

```
class UserService {  
    // Get data from database  
    public User getUser() {  
        return null;  
    }  
  
    // Check validation  
    public boolean isValid() {  
        return true;  
    }  
  
    // Show Notification  
    public void showNotification() {  
  
    }  
  
    // Logging  
    public void logging() {  
        System.out.println("...");  
    }  
  
    // Parsing  
    public User parseJson(String json) {  
        return null;  
    }  
}
```

- Each student gives 01 case of SRP violation and proposes a solution.

SRP: Exercise 2



Design & implement the following context to satisfy SRP:

- Let's consider a hotel. This hotel has a hotel keeper. There are a lot of restaurants inside the hotel e.g. Vegetarian restaurants, Non-Vegetarian restaurants, and Omnivore restaurants.
- You, as a client want access to different menus of different restaurants. You do not know what are the different menus they have. You just have access to a hotel keeper who knows his hotel well.
- Whichever menu you want, you tell the hotel keeper and he takes it out of the respective restaurants and hands it over to you.

CHAPTER 2. SOLID Principles



2.1. **Single Responsibility Principle**

2.2. **Open-Closed Principle**

2.3. **Liskov Substitution Principle**

2.4. **Interface Segregation Principle**

2.5. **Dependency Inversion Principle**

OCP: The Open-Closed Principle

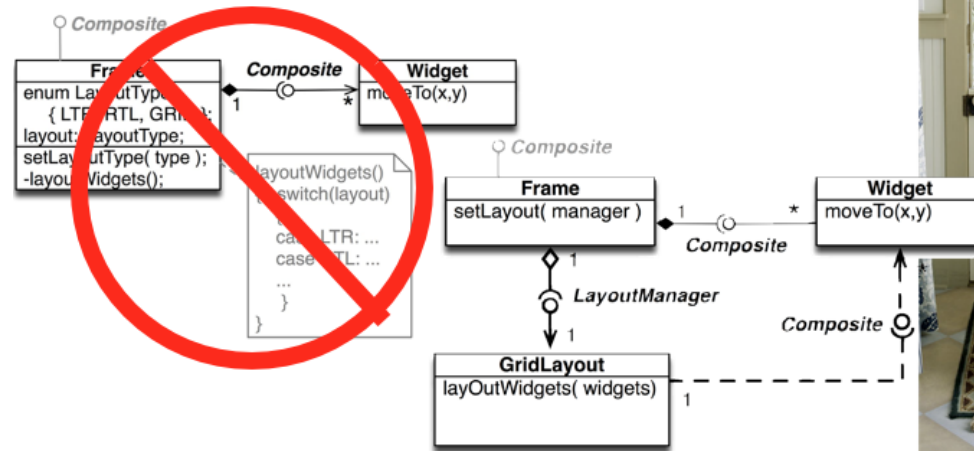


The Open Closed Principle (OCP) was coined in 1988 by Bertrand Meyer. It says:

- *A software artifact should be open for extension but closed for modification.*

**Open for extension;
Closed for modification.**

Changes shouldn't ripple out to dependent classes/objects/modules



OCP: The Open-Closed Principle



Most students of software design recognize the OCP as a principle that guides them in the design of classes and modules. But the principle takes on even greater significance when we consider the level of architectural components.

OCP: Case-study



- We have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.
- The stakeholders ask that this same information be turned into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.

OCP: Case-study



- We have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.
- The stakeholders ask that this same information be turned into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.
- Clearly, some new code must be written. But how much old code will have to change?

OCP: Case-study



- Clearly, some new code must be written. But how much old code will have to change?
- A good software architecture would reduce the amount of changed code to the barest minimum. Ideally, **zero**.
- How? By properly separating the things that change for different reasons (the Single Responsibility Principle), and then organizing the dependencies between those things properly (the Dependency Inversion Principle).

OCP: Case-study



Applying the SRP

OCP: Case-study



Applying the SRP

- The essential insight here is that generating the report involves two separate responsibilities: the calculation of the reported data, and the presentation of that data into a web- and printer-friendly form.
- Having made this separation, we need to organize the source code dependencies to ensure that changes to one of those responsibilities do not cause changes in the other. Also, the new organization should ensure that the behavior can be extended without undo modification.

OCP: Solution (1)



If component A should be **protected from changes** in component B, then component B should **depend on** component A.

Eg: From above analyses, we can design which components are “equal”, which are higher ones, which are protected from changes from which ones, and which ones depend on others.

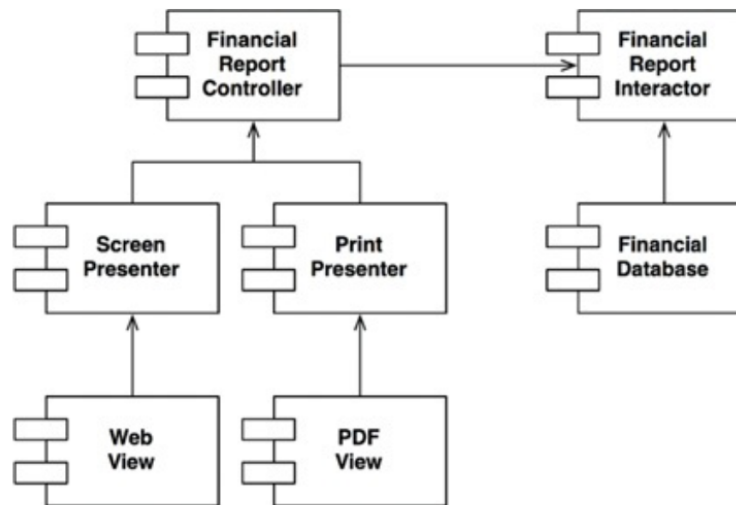


Figure 8.3 The component relationships are unidirectional

OCP: Solution (1)



- The *Interactor* is in the position that best conforms to the OCP. Changes to the *Database*, or the *Controller*, or the *Presenters*, or the *Views*, will have no impact on the *Interactor*.

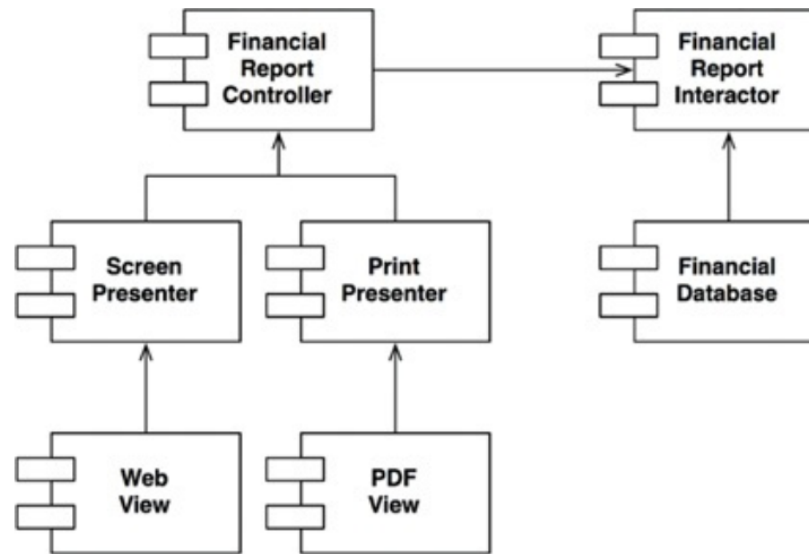


Figure 8.3 The component relationships are unidirectional

- This is how the OCP works at the architectural level. Architects separate functionality based on how, why, and when it changes.

OCP: Solution (1)

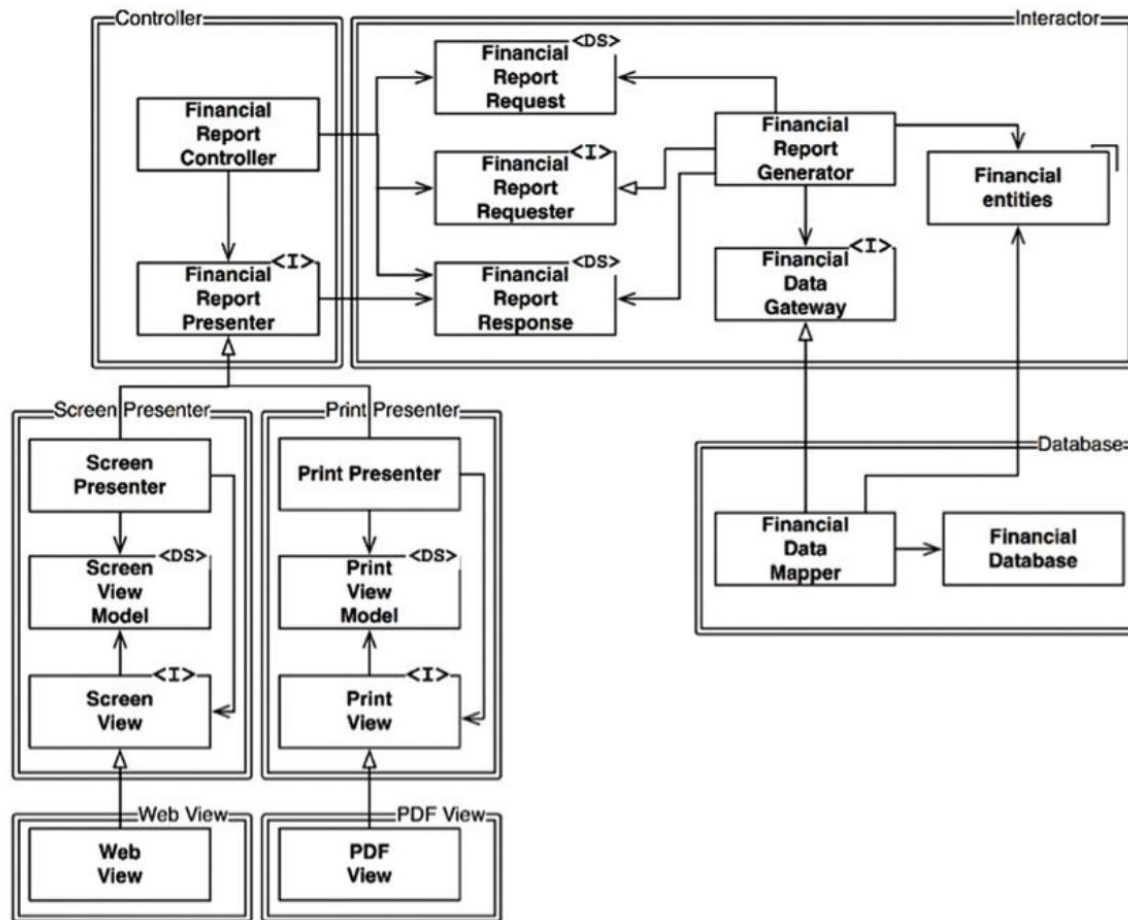


Figure - Partitioning the processes into classes and separating the classes into components

OCP: Solution (1)



- The OCP is one of the driving forces behind the architecture of systems. The goal is to make the system **easy to extend without incurring a high impact of change**. This goal is accomplished by partitioning the system into **components**, and arranging those components into a dependency hierarchy that **protects** higher-level components from changes in lower-level components.

OCP: Solution (2) - Strategy Pattern

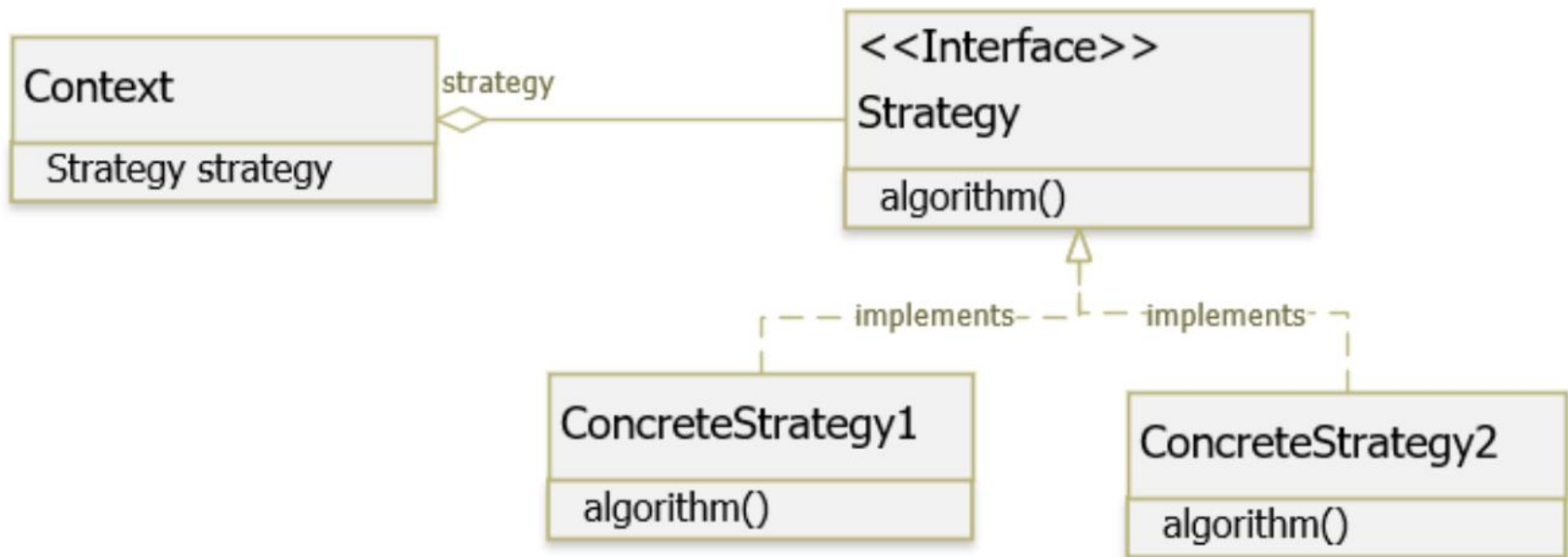


- System components (classes, modules, functions) should be **open for extension** but **closed for modification**. That is, you should be able to extend the functionality of a component without changing its existing source code. This can be achieved through inheritance, using interfaces, or design patterns like the **Strategy Pattern**.
- Eg: instead of modifying a class directly to add new functionality, you can extend it through inheritance.

OCP: Solution (2) - Strategy Pattern



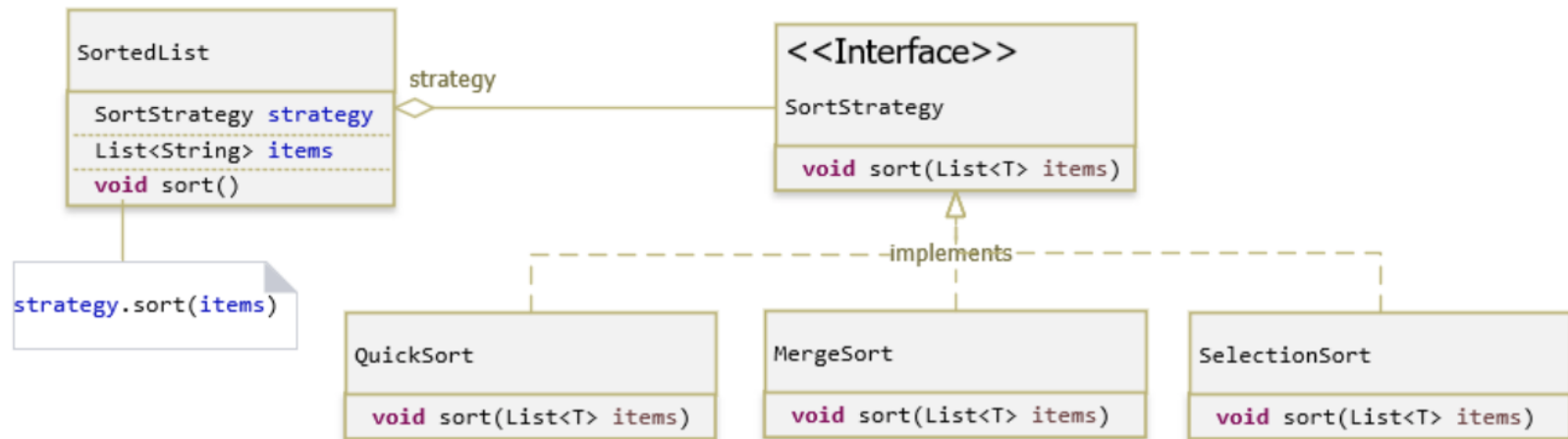
- Behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects.



OCP: Solution (2) - Strategy Pattern



- Example:



OCP: Exercise 1

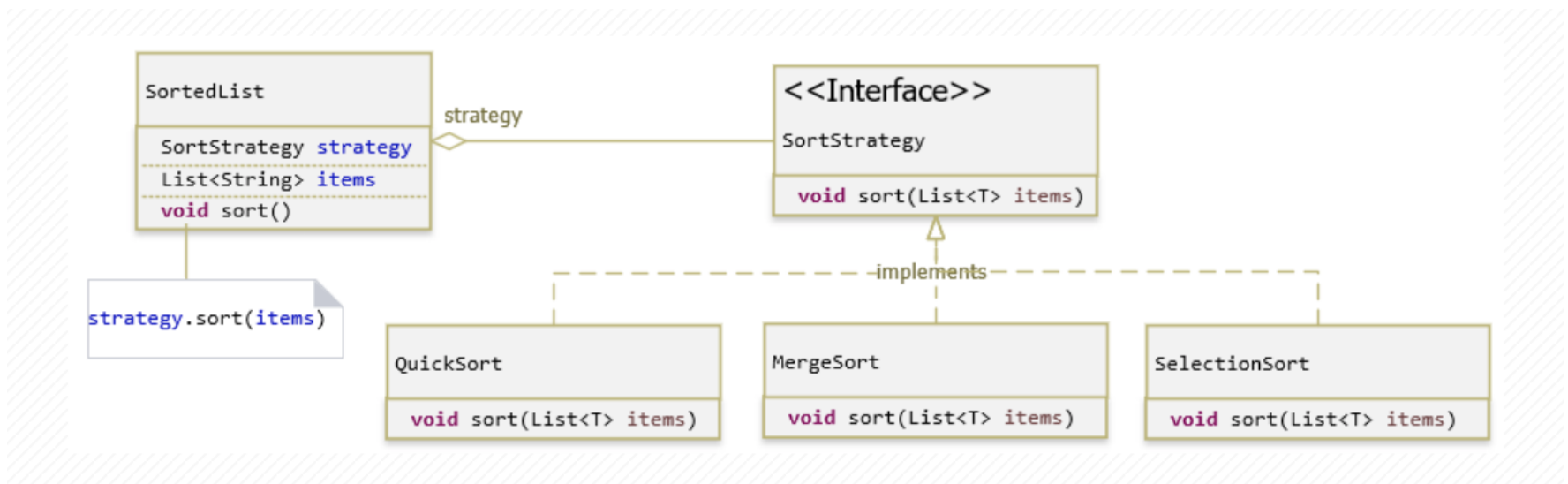


- Each student gives 01 case of OCP violation and refactors it.

OCP: Exercise 2



- Students implement the following Strategy pattern for sorting algorithms by an OOP programming language, such as Java.



CHAPTER 2. SOLID Principles



2.1. **Single Responsibility Principle**

2.2. **Open-Closed Principle**

2.3. **Liskov Substitution Principle**

2.4. **Interface Segregation Principle**

2.5. **Dependency Inversion Principle**

LSP: The Liskov Substitution Principle

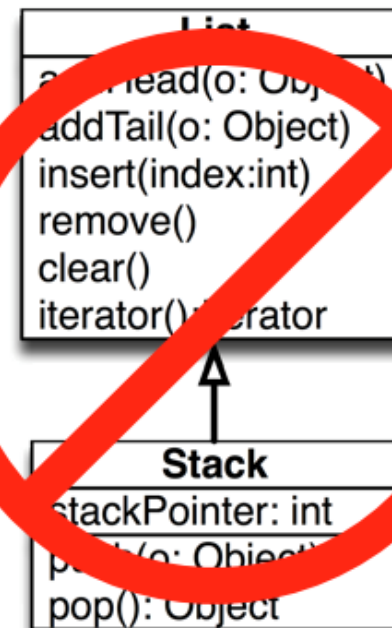


Liskov Substitution Principle

Subtypes must be substitutable for their base types.

```
f( List l )
{
    l.clear();
    l.addHead("x");
}
Stack aStack...;
f(aStack);
```

Context Matters!



LSP: The Liskov Substitution Principle



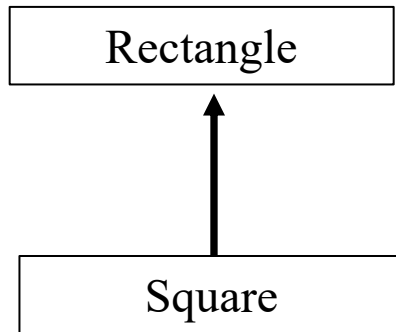
- Objects of a **child class** must be able to replace the **parent class** without changing the correctness of the program. In other words, the child class must **maintain the behavior** of the parent class and not change the logic **when replacing it**.

LSP: The Liskov Substitution Principle



- In the early years of the object-oriented revolution, we thought of the LSP as a way to guide the use of inheritance, as shown in the previous sections. However, over the years the LSP has morphed into a broader principle of software design that pertains to interfaces and implementations.

LSP: Violation



```
40 public class LSPEXample1 {
41     public void example1() {
42         Rectangle rect = new Rectangle();
43         rect.setWidth(5);
44         rect.setHeight(10);
45         System.out.println(rect.calculateArea()); // 50
46
47         Square square = new Square();
48         square.setWidth(5);
49         square.setHeight(10);
50         System.out.println(square.calculateArea()); // 100
51     }
52 }
```

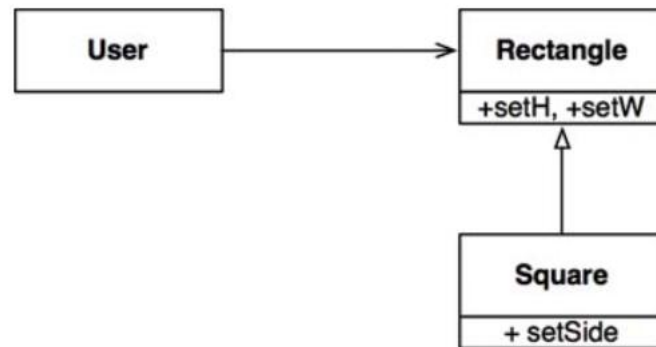


```
1 package com.gpcoder.solid;
2
3 /**
4  * LSP - Liskov substitution principle example
5  *
6  * @author gpcoder
7  */
8 class Rectangle {
9     private int width;
10    private int height;
11
12    public int calculateArea() {
13        return this.width * this.height;
14    }
15
16    public void setWidth(int width) {
17        this.width = width;
18    }
19
20    public void setHeight(int height) {
21        this.height = height;
22    }
23 }
24
25 class Square extends Rectangle {
26
27     @Override
28     public void setWidth(int width) {
29         super.setWidth(width);
30         super.setHeight(width);
31     }
32
33     @Override
34     public void setHeight(int height) {
35         super.setWidth(height);
36         super.setHeight(height);
37     }
38 }
```

LSP: Violation



- Example LSP Violation



- In this example, Square is not a proper subtype of Rectangle because the height and width of the Rectangle are independently mutable; in contrast, the height and width of the Square must change together. Since the User believes it is communicating with a Rectangle, it could easily get confused.

LSP: Example



- Imagine that we have a class named License, as shown in the following figure. This class has a method named calcFee(), which is called by the Billing application. There are two “subtypes” of License: PersonalLicense and BusinessLicense. They use different algorithms to calculate the license fee.
- The following design conforms to the LSP because the behavior of the Billing application does not depend, in any way, on which of the two subtypes it uses. Both of the subtypes are substitutable for the License type.

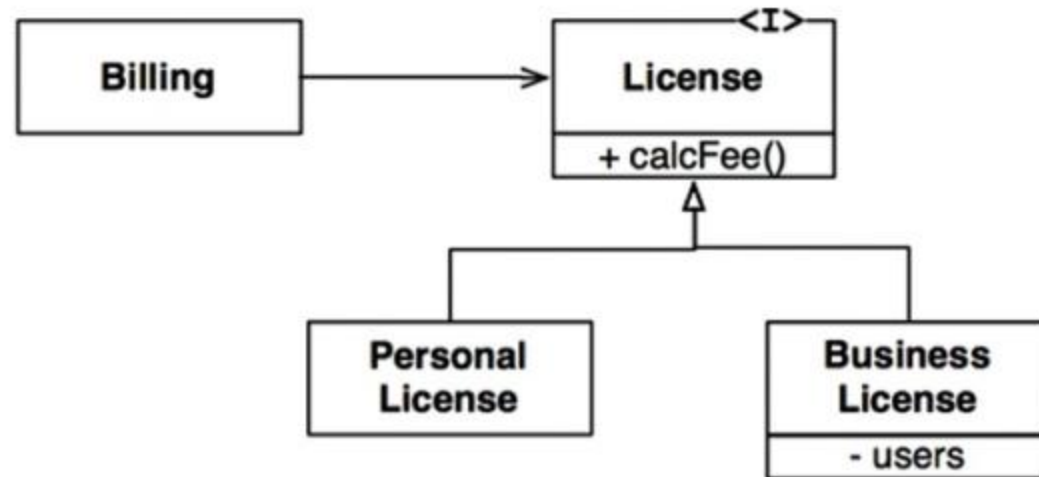


Figure 9.1 License, and its derivatives, conform to LSP

LSP: Conclusion



- The LSP can, and should, be extended to the level of architecture. A simple violation of substitutability, can cause a system's architecture to be polluted with a significant amount of extra mechanisms.

LSP: Exercise



- Each student gives 01 case of LSP violation and refactors it.

CHAPTER 2. SOLID Principles



2.1. **Single Responsibility Principle**

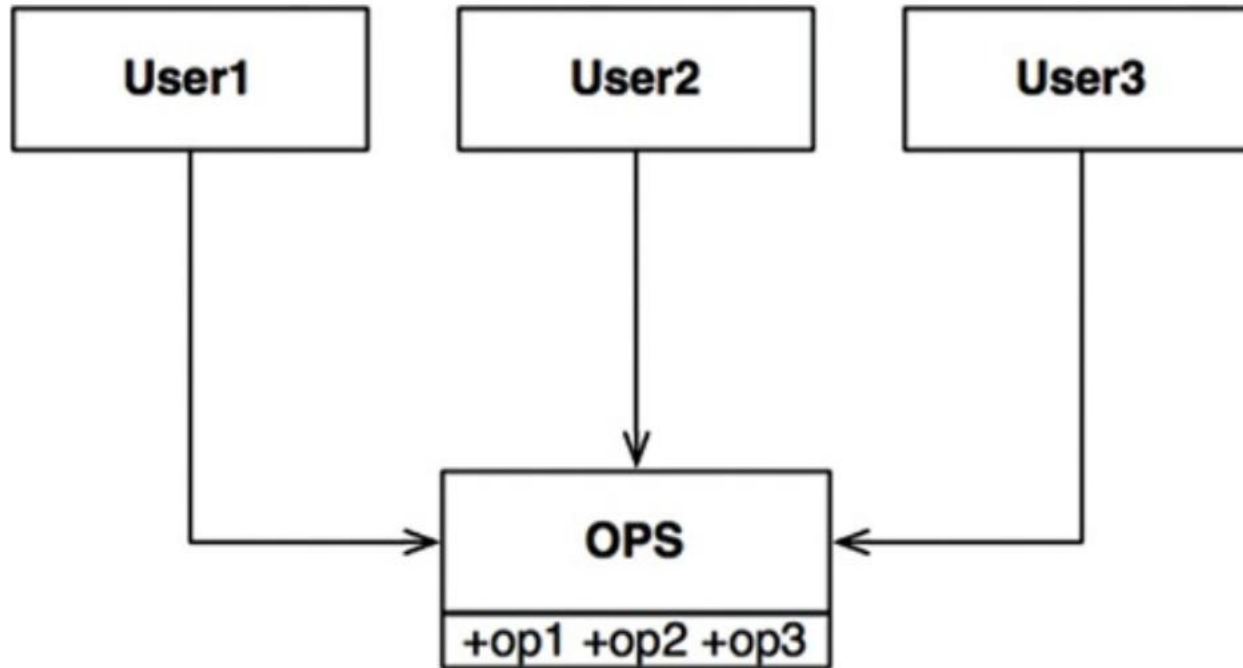
2.2. **Open-Closed Principle**

2.3. **Liskov Substitution Principle**

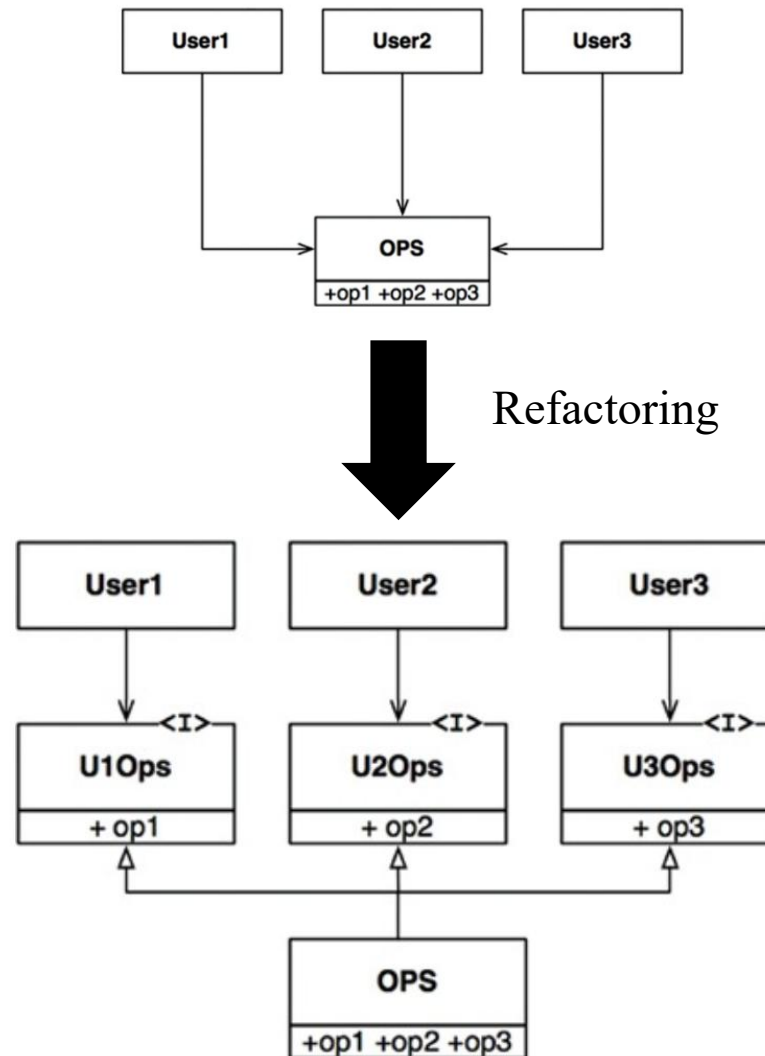
2.4. **Interface Segregation Principle**

2.5. **Dependency Inversion Principle**

ISP: The Interface Segregation Principle



ISP: The Interface Segregation Principle



ISP: Example



```
public interface Animal {  
    void Eat();  
    void Drink();  
    void Sleep();  
    //...  
    void Swim();  
    void Fly();  
}
```

```
public class Cat implements Animal{  
    public void Eat() {  
    }  
    public void Drink() {  
    }  
    public void Sleep() {  
    }  
    @Override  
    public void Swim() {  
        // TODO Auto-generated method stub  
    }  
    @Override  
    public void Fly() {  
        // TODO Auto-generated method stub  
    }  
}
```

Class Cat doesn't have Swim() and Fly() method!

ISP: Example



- Segregate into 2 interfaces: Animal and Bird

```
public interface Animal {  
    void Eat();  
    void Drink();  
    void Sleep();  
}
```

```
2 public interface Bird {  
3     void Fly();  
4 }  
5
```

- Each subclass implements necessary interfaces itself.

```
2 public class Cat implements Animal{  
3  
4     @Override  
5     public void Eat() {  
6         // TODO Auto-generated method stub  
7  
8     }  
9  
10    @Override  
11    public void Drink() {  
12        // TODO Auto-generated method stub  
13  
14    }  
15  
16    @Override  
17    public void Sleep() {  
18        // TODO Auto-generated method stub  
19  
20    }  
21 }  
22
```

```
2 public class Dove implements Animal, Bird  
3 {  
4  
5     public void Eat() {..  
6  
7     }  
8     public void Drink() {..  
9  
10    }  
11    public void Sleep() {..  
12  
13    }  
14  
15    @Override  
16    public void Fly() {  
17        // TODO Auto-generated method stub  
18  
19    }  
20 }  
21
```

ISP: The Interface Segregation Principle



- **ISP and language**

- Statically typed languages like Java force programmers to create declarations that users must *import*, or *use*, or otherwise *include*. It is these included declarations in source code that create the source code dependencies that force recompilation and redeployment.
- In dynamically typed languages like Ruby and Python, such declarations don't exist in source code. Instead, they are inferred at runtime. Thus there are no source code dependencies to force recompilation and redeployment.

ISP: The Interface Segregation Principle



- ISP and architecture

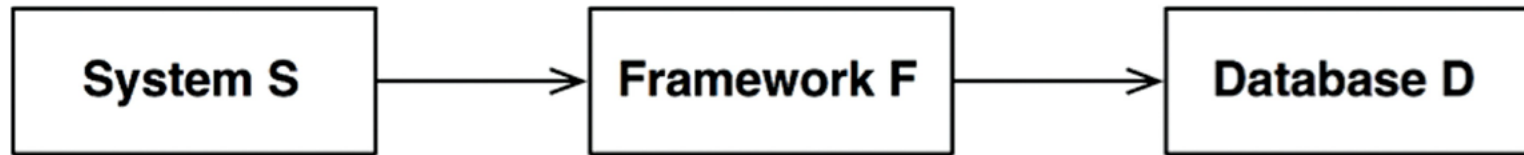


Figure 10.3 A problematic architecture

- Suppose that D contains features that F does not use and, therefore, that S does not care about. Changes to those features within D may well force the redeployment of F and, therefore, the redeployment of S. Even worse, a failure of one of the features within D may cause failures in F and S.
=> In general, it is harmful to depend on modules that contain more than you need.

ISP: Conclusion



- The lesson here is that depending on something that carries baggage that you don't need can cause you troubles that you didn't expect.
- Interfaces should be small and specific, so that classes only need to implement the methods they really need. Avoid creating large interfaces with lots of unrelated methods that the class doesn't use.

ISP: Exercise



You are designing a system for managing **office devices**. The following interface is currently used:

```
public interface OfficeDevice {  
    void print();  
    void scan();  
    void fax();  
}
```

- Several devices implement this interface:
 - Printer supports **print** only
 - Scanner supports **scan** only
 - MultiFunctionPrinter supports **print, scan, fax**
- Questions
 - Does the current design violate the Interface Segregation Principle (ISP)? Why?
 - If so, please redesign the system to comply with ISP.

ISP: Exercise



- Each student gives 01 case of ISP violation and refactors it.

CHAPTER 2. SOLID Principles



2.1. **Single Responsibility Principle**

2.2. **Open-Closed Principle**

2.3. **Liskov Substitution Principle**

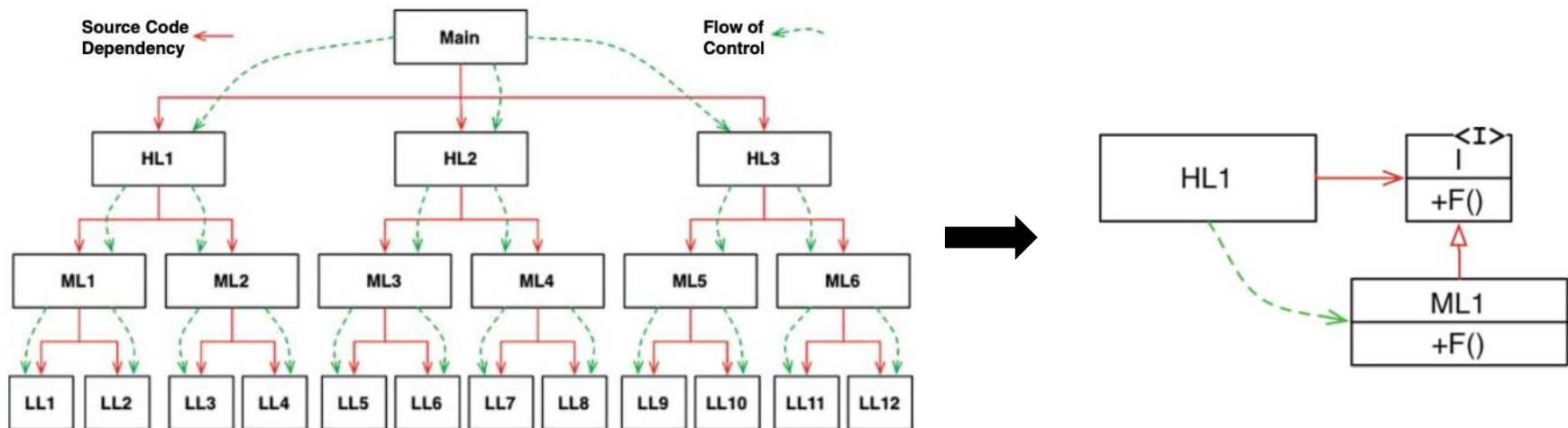
2.4. **Interface Segregation Principle**

2.5. **Dependency Inversion Principle**

DIP: The Dependency Inversion Principle



The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code **dependencies** refer only to **abstractions**, not to **concretions**.

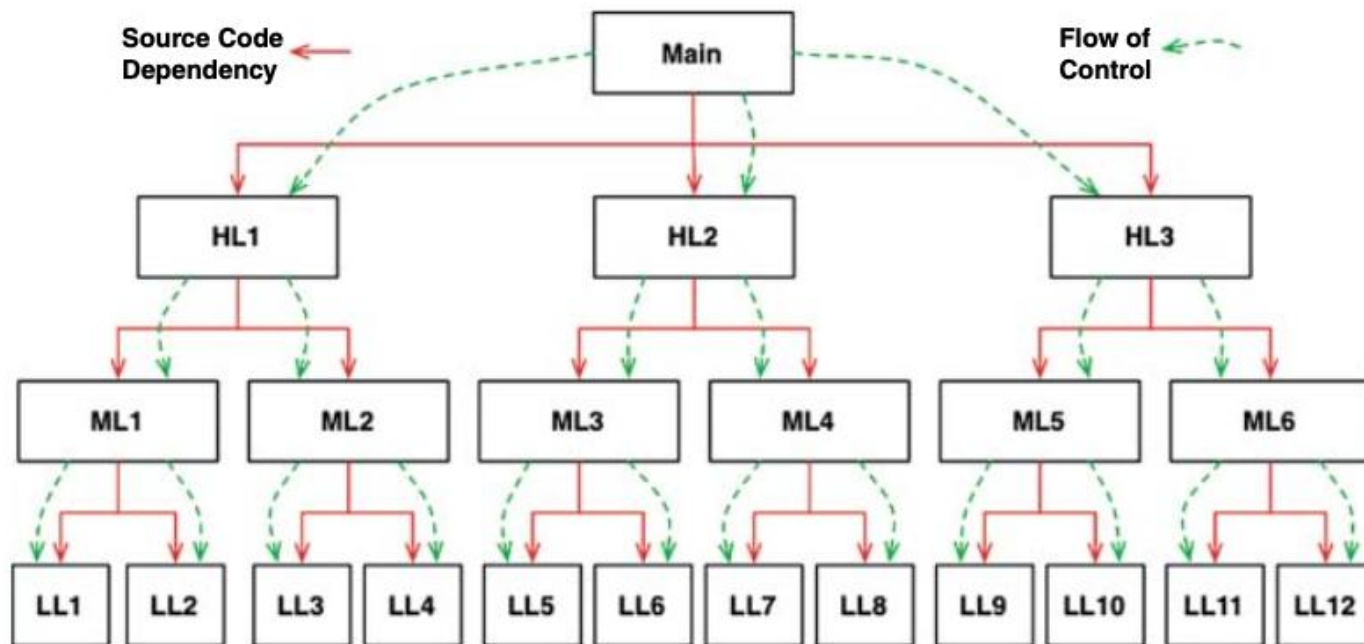


DIP: The Dependency Inversion Principle

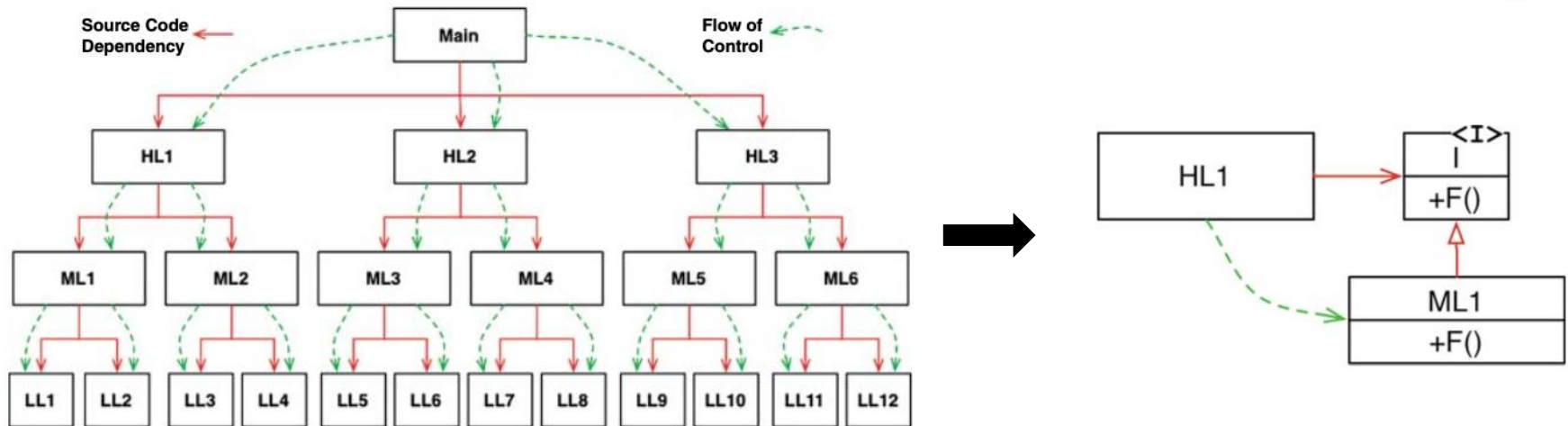


What is Dependency Inversion?

- In the typical calling tree, main functions called high-level functions, which called mid-level functions, which called low-level functions. In that calling tree, however, source code dependencies inevitably followed the flow of control.

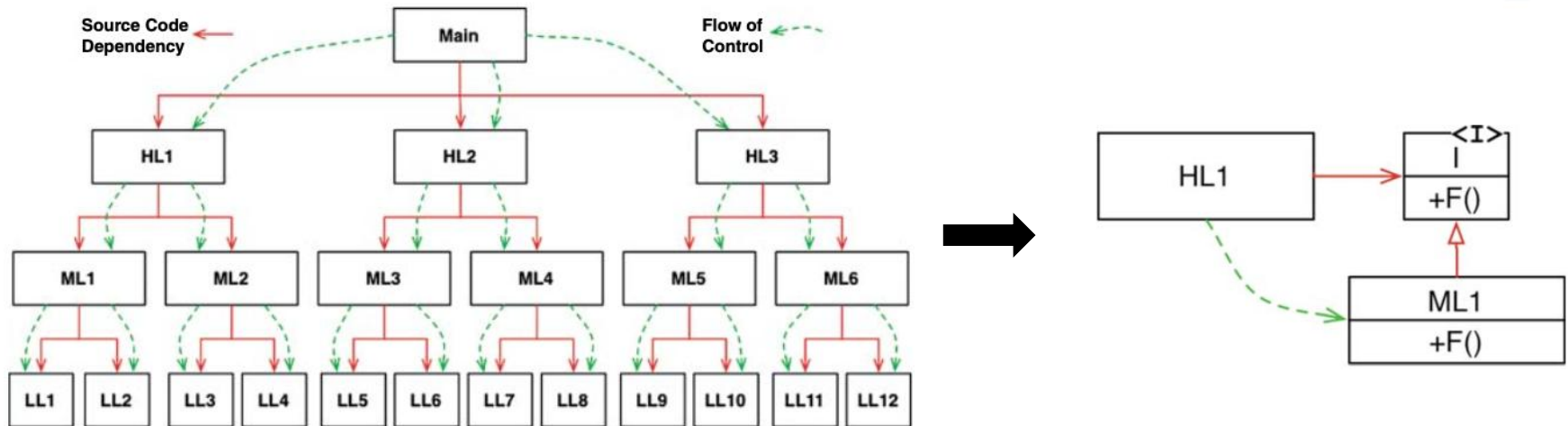


DIP: The Dependency Inversion Principle



- In the right part of the figure above, module HL1 calls the F() function in module ML1. The fact that it calls this function through an interface is a source code contrivance. At runtime, the interface doesn't exist. HL1 simply calls F() within ML1.

DIP: The Dependency Inversion Principle



- In fact, the flow of control was decided by the behavior of the system, and the source code dependencies were decided by that flow of control.
 - The source code dependency between ML1 and the interface I points in the opposite direction compared to the flow of control.
- => Dependency inversion.
- *In OOP, any source code dependency, no matter where it is, can be inverted.*

DIP: The Dependency Inversion Principle



The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code **dependencies** refer only to **abstractions**, not to **concretions**.

- *High-level modules should not depend on low-level modules. Both should depend on abstractions.*
- *Abstractions should not depend upon details. Details should depend upon abstractions.*

DIP: Stable Abstractions



- Every change to an abstract interface corresponds to a change to its concrete implementations. Conversely, changes to concrete implementations do not always, or even usually, require changes to the interfaces that they implement. Therefore, **interfaces are less volatile than implementations.**
- Good software designers and architects work hard to **reduce the volatility of interfaces.** They try to find ways to add functionality to implementations without making changes to the interfaces.

DIP: The Dependency Inversion Principle



- **Coding practices:**
 - **Don't refer to volatile concrete classes.** Refer to abstract interfaces instead. It also puts severe constraints on the creation of objects and generally enforces the use of Abstract Factories.
 - **Don't derive from volatile concrete classes.** This is a corollary to the previous rule, but it bears special mention. That focus on using inheritance relationships with great care.
 - **Don't override concrete functions.** Concrete functions often require source code dependencies. To manage those dependencies, you should make the function abstract and create multiple implementations.
 - **Never mention the name of anything concrete and volatile.** This is really just a restatement of the principle itself.

DIP: Abstract Factory

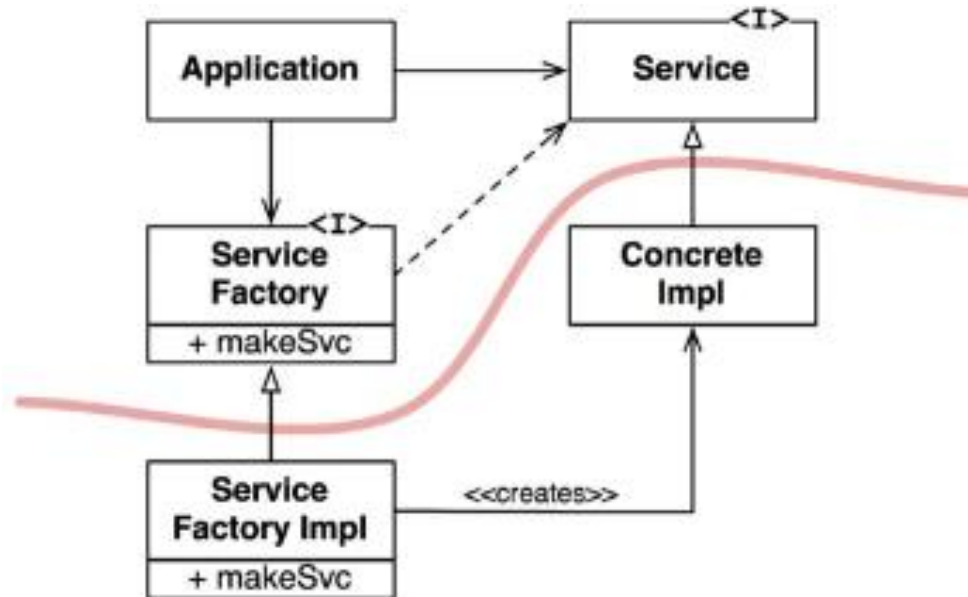


- To comply with these rules, the creation of volatile concrete objects requires special handling. This caution is warranted because, in virtually all languages, the creation of an object requires a source code dependency on the concrete definition of that object.
- In most object-oriented languages, such as Java, we would use an **Abstract Factory** to manage this undesirable dependency.

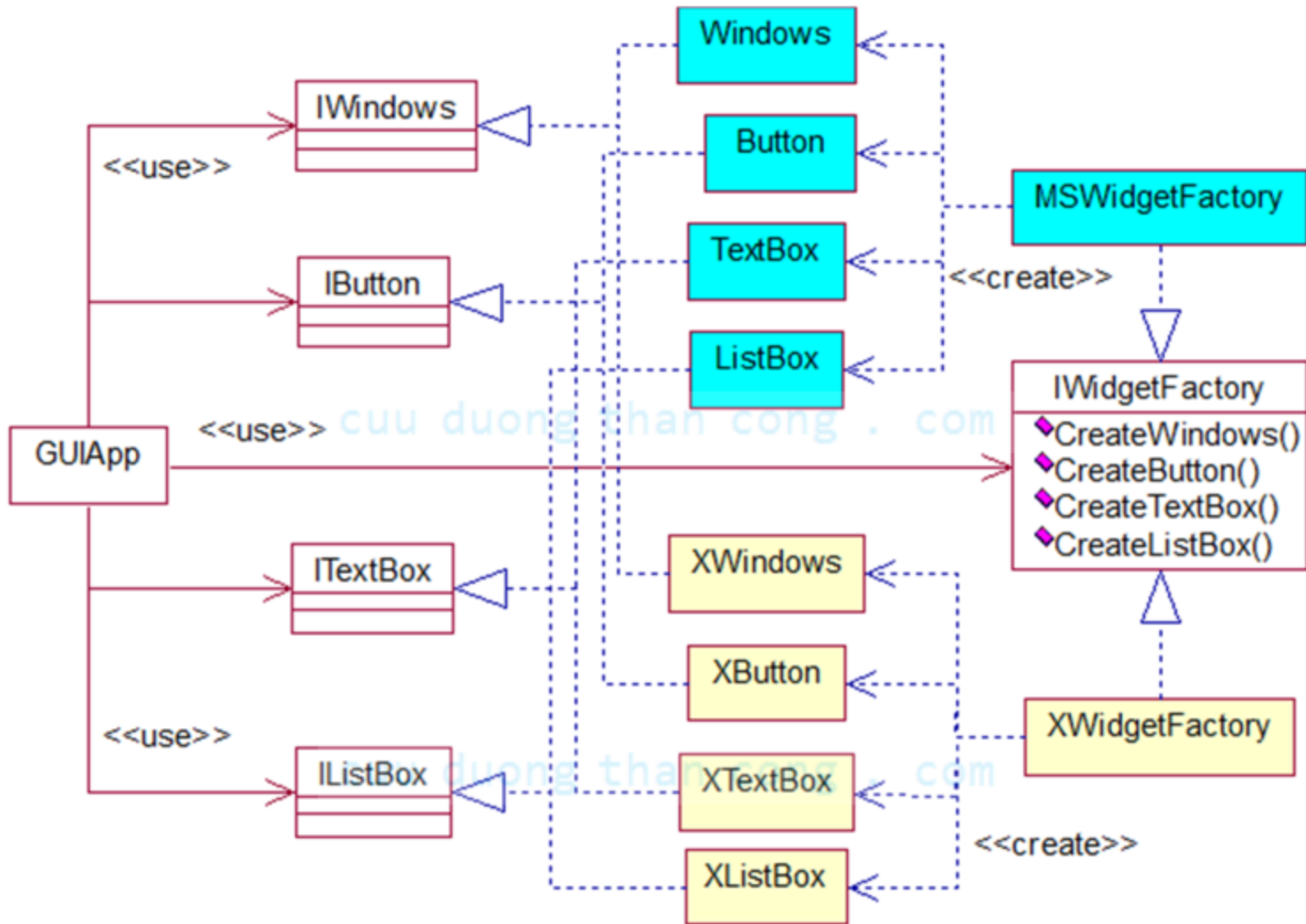
DIP: Abstract Factory



- E.g.: Use of the Abstract Factory pattern to manage the dependency.
- The (red) curved line is an architectural boundary. It separates the abstract from the concrete. All source code dependencies cross that curved line pointing in the same direction, toward the abstract side.



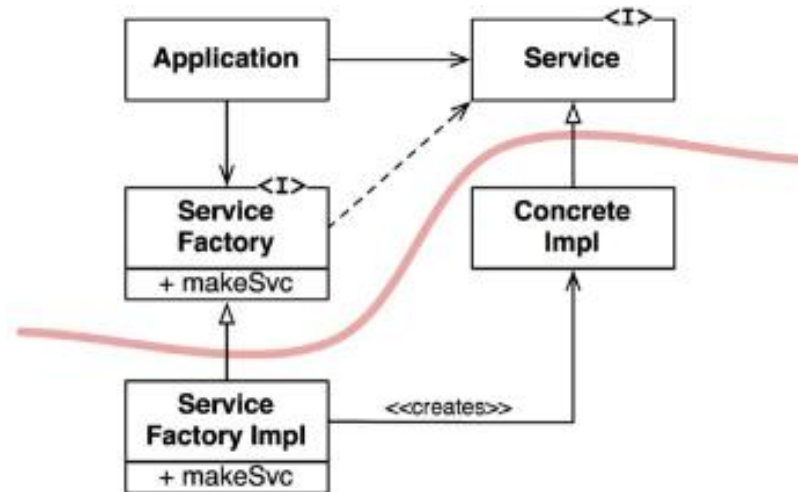
DIP: Example



DIP: Conclusion



- As we have more higher-level architectural components, DIP will be the most visible organizing principle in our architecture diagrams. The curved line (as figure) will become the architectural boundaries. The way the **dependencies cross that curved line in one direction**, and toward more abstract entities, will become a new rule that we will call the **Dependency Rule**.



DIP: Exercise



Each You are developing a **cross-platform notification system** for an enterprise application. The system must support different **platforms** (e.g., Web and Mobile).

Each platform requires a **consistent family of notification services**:

- Email notification**

- SMS notification**

The initial implementation is shown below:

```
public class WebEmailSender {  
    public void sendEmail(String message) {  
        System.out.println("Sending Email via Web: " + message);  
    }  
}
```


DIP: Exercise



```
public class WebSmsSender {  
    public void sendSms(String message) {  
        System.out.println("Sending SMS via Web: " + message);  
    }  
}
```

```
public class NotificationService {  
    private WebEmailSender emailSender = new WebEmailSender();  
    private WebSmsSender smsSender = new WebSmsSender();  
  
    public void notifyUser(String message) {  
        emailSender.sendEmail(message);  
        smsSender.sendSms(message);  
    }  
}
```

DIP: Exercise



- **Questions:**
 - Explain why the current design violates the **Dependency Inversion Principle (DIP)**.
 - Redesign the system using **Abstract Factory** so that it complies with DIP.
 - Explain how Abstract Factory helps enforce DIP in this design.

DIP: Exercise



- Each student gives 01 case of DIP violation and refactors it.

Conclusion



Q&A